

Programación Orientada a Objetos

Seguimiento # 1

Actividad individual valor de 10%

Estudiante

John Mario Mejía Arango

1017243551 cc

Docente

Walter Hugo Arboleda Mazo

Universidad Nacional de Colombia

Sede Medellín

Medellín, Antioquia

17/09/2026

Este es el taller de seguimiento realizado por el estudiante John Mario Mejía Arango, cada punto fue realizado en google colab.

Punto # 4

<https://colab.research.google.com/drive/1hEXQbrGLXrAWXUF-PCocEUHERDCkuVql?usp=sharing>

En el pasado link puede encontrar la solución en google colab, a continuación, adjunto el código.

```
from dataclasses import dataclass

def _fmt(n: float) -> str:
    return str(int(n)) if float(n).is_integer() else f"{n:.2f}"

@dataclass
class Persona:
    nombre: str
    edad: float

class FamiliaJuan:
    def __init__(self, edad_juan: float):
        self.juan = Persona("Juan", edad_juan)
        self.alberto = Persona("Alberto", 2 * (edad_juan / 3))
        self.ana = Persona("Ana", 4 * (edad_juan / 3))
        self.mama = Persona("Mamá", self.juan.edad + self.alberto.edad
+ self.ana.edad)

    def resumen(self) -> list[str]:
        return [
            f"Juan: {_fmt(self.juan.edad)}",
            f"Ana: {_fmt(self.ana.edad)}",
            f"Alberto: {_fmt(self.alberto.edad)}",
            f"Mamá: {_fmt(self.mama.edad)}",
        ]

def main():
    try:
        edjuan = float(input("Ingrese la edad de Juan: ").strip())
    except ValueError:
        print("Entrada inválida. Debe ser un número.")
    return
```

```

if not (0 < edjuan <= 120):
    print("La edad debe estar entre 1 y 120 años.")
    return

familia = FamiliaJuan(edjuan)
print("LAS EDADES SON:")
for linea in familia.resumen():
    print(linea)

if __name__ == "__main__":
    main()

```

Punto # 5

https://colab.research.google.com/drive/1SHk2QSE_7dLwTpLprjIOEixNKjeQrLw6?usp=sharing

En el pasado link puede encontrar la solución en google colab, a continuación, adjunto el código.

```

from dataclasses import dataclass

def _fmt(n: float) -> str:
    return str(int(n)) if float(n).is_integer() else f"{n:.2f}"

@dataclass
class Estado:
    paso: str
    suma: float
    x: float
    y: float

def ejecutar(x: float, y: float) -> tuple[float, list[Estado]]:
    pasos: list[Estado] = []
    suma = 0.0
    pasos.append(Estado("Inicio", suma, x, y))

    suma += x
    pasos.append(Estado("suma = suma + x", suma, x, y))

    x = x + y ** 2
    pasos.append(Estado("x = x + y**2", suma, x, y))

```

```

    suma += x / y
    pasos.append(Estado("suma = suma + x/y", suma, x, y))

    return suma, pasos

def main():
    try:
        x = float(input("Ingrese X: ").strip())
        y = float(input("Ingrese Y: ").strip())
    except ValueError:
        print("Entrada inválida. Deben ser números.")
        return

    if y == 0:
        print("Y no puede ser 0 (división por cero).")
        return

    resultado, pasos = ejecutar(x, y)

    print("Prueba de escritorio")
    for p in pasos:
        print(f"{p.paso:22} -> SUMA={_fmt(p.suma)} X={_fmt(p.x)} Y={_fmt(p.y)}")

    print(f"\nEL VALOR DE LA SUMA ES: {_fmt(resultado)}")

if __name__ == "__main__":
    main()

```

Punto # 12

<https://colab.research.google.com/drive/1LWXNYvMdv1m3HxREBmU89NyEXnxFO-GS?usp=sharing>

En el pasado link puede encontrar la solución en google colab, a continuación, adjunto el código.

```

from dataclasses import dataclass

def _fmt_money(v: float) -> str:
    return (f"${v:,.2f}" # 1,234.56
            .replace(",", "_")
            .replace(".", ","))

```

```

        .replace("_", "."))

def _read_float(prompt: str, default: float) -> float:
    raw = input(f"{prompt} [{default}]: ").strip()
    return default if raw == "" else float(raw)

@dataclass
class Nomina:
    horas: float
    tarifa_hora: float
    retencion_pct: float # porcentaje, ej. 12.5

    @property
    def salario_bruto(self) -> float:
        return self.horas * self.tarifa_hora

    @property
    def retencion_fuente(self) -> float:
        return self.salario_bruto * (self.retencion_pct / 100.0)

    @property
    def salario_netto(self) -> float:
        return self.salario_bruto - self.retencion_fuente

def main():
    try:
        horas = _read_float("Horas trabajadas", 48)
        tarifa = _read_float("Tarifa por hora ($)", 5000)
        pct = _read_float("% Retención en la fuente", 12.5)
    except ValueError:
        print("Entrada inválida. Use números.")
        return

    if horas < 0 or tarifa < 0 or not (0 <= pct <= 100):
        print("Valores fuera de rango: horas/tarifa no negativas y %
entre 0 y 100.")
        return

    n = Nomina(horas, tarifa, pct)

    print("Resultados:")
    print(f"Salario bruto: {_fmt_money(n.salario_bruto)}")
    print(f"Retención en la fuente: {_fmt_money(n.retencion_fuente)}")

```

```

    print(f"Salario neto: {_fmt_money(n.salarario_neto)}")

if __name__ == "__main__":
    main()

```

Punto # 14

<https://colab.research.google.com/drive/1LWXNYvMdv1m3HxREBmU89NyEXnxFO-GS?usp=sharing>

En el pasado link puede encontrar la solución en google colab, a continuación, adjunto el código.

```

def main():
    try:
        n = float(input("Ingrese un número: ").strip())
    except ValueError:
        print("Entrada inválida. Debe ser un número.")
        return

    cuadrado = n ** 2
    cubo = n ** 3

    print(f"Cuadrado: {cuadrado}")
    print(f"Cubo: {cubo}")

if __name__ == "__main__":
    main()

```

Punto # 17

<https://colab.research.google.com/drive/1HTFA-gyiKZPFZ7UwiBiSWfTopWFXqfFi?usp=sharing>

En el pasado link puede encontrar la solución en google colab, a continuación, adjunto el código.

```

from math import pi

def main():
    try:
        r = float(input("Ingrese el radio del círculo: ").strip())
    except ValueError:
        print("Entrada inválida. Debe ser un número.")
        return

```

```
if r < 0:
    print("El radio no puede ser negativo.")
    return

area = pi * (r ** 2)
longitud = 2 * pi * r

print(f"Área del círculo: {area}")
print(f"Longitud de la circunferencia: {longitud}")

if __name__ == "__main__":
    main()
```